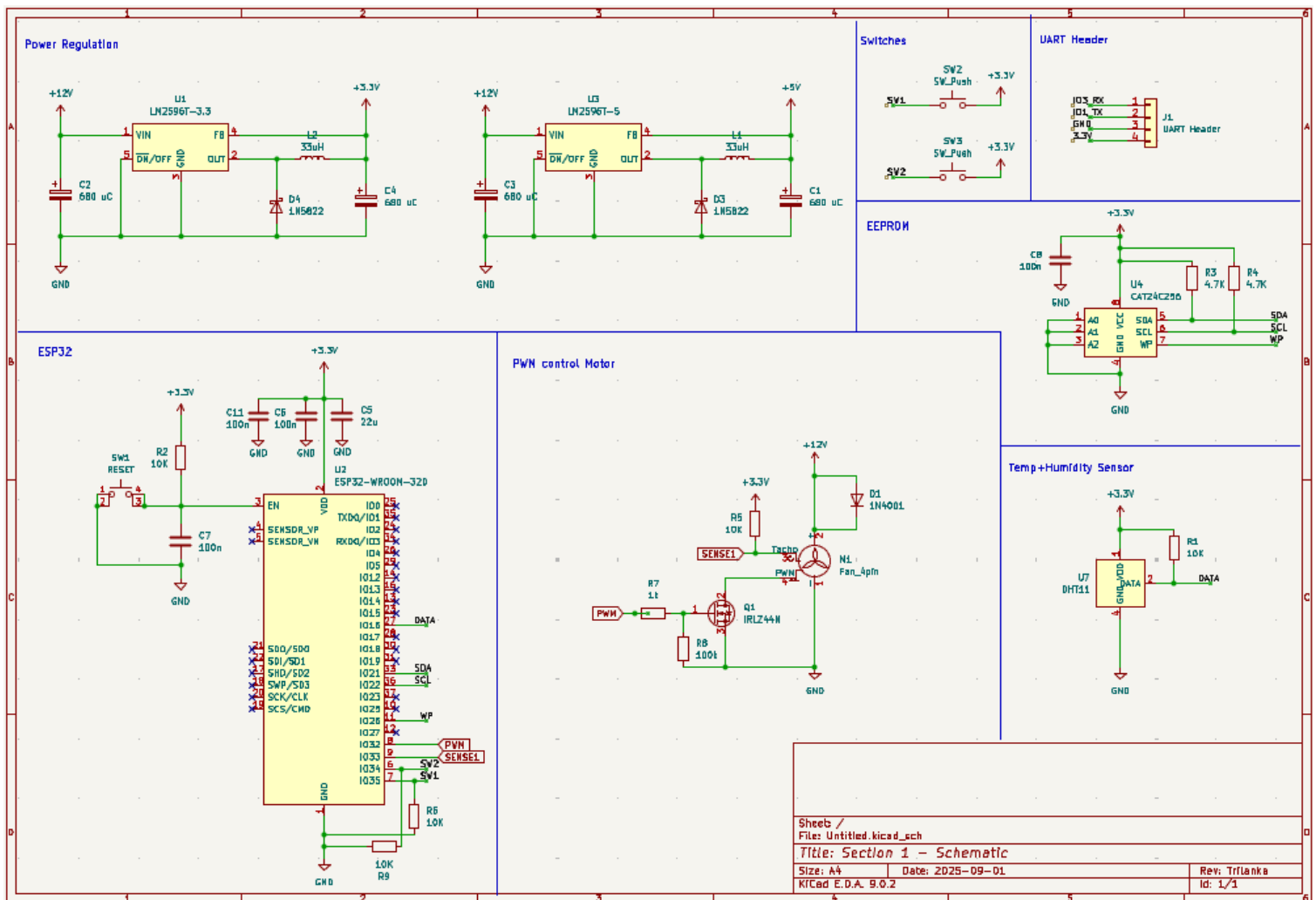


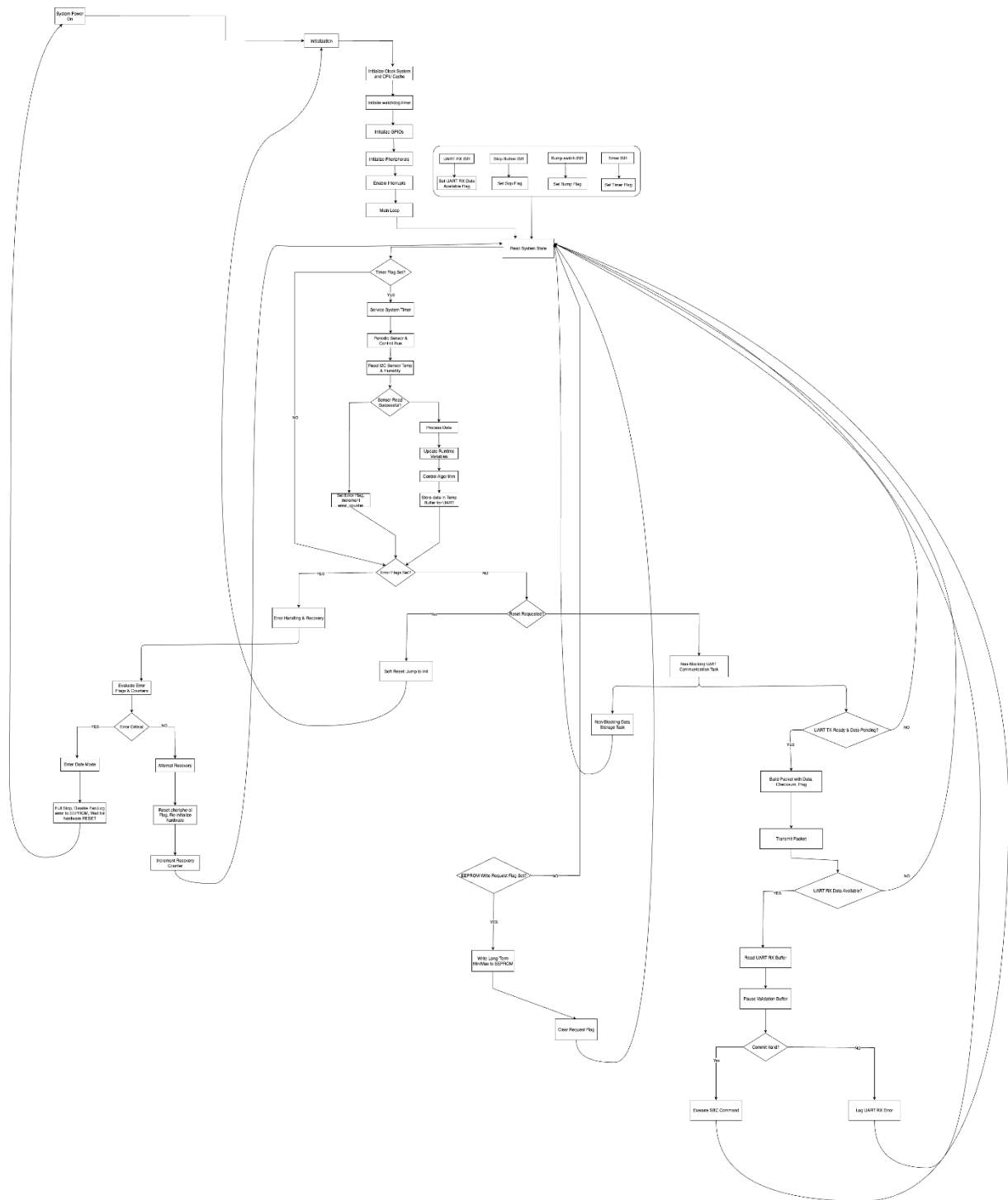
Section 1 – Electronics & Embedded Systems

Knowledge Task

01. Schematic & PCB Concept



Important: I have attached the UART header as GPIO pins and power pins according to the ESP32 side. When making the PCB, we need to label the pins as RX → TX, TX → RX, and 3.3V as output. Please find the Google Drive link to get the 'kiCAD' file for this schematic.



The above flow-chart handles Initialization, Sensor processing, control logic, Data storing in EEPROM, UART communication, Interrupt handling and the error handling paths. I have used FAN as the actuator here. To be more precise I have considered Error Flags for the Sensor reading, FAN control, Start/Stop/Reset Buttons and other usual error flags (e.g. – Timer) as well.

In my architecture after initialization part we enter to the main Loop and inside that loop we handle those system states (Flag states).

Since the above image is not enough visible, I have attached MyDrive Link where you can find resources I have used in this document.

Link: <https://drive.google.com/drive/folders/1PrFsNwHKQePNxLUsgWG4FIElJa5FY1dd?usp=sharing>

03. Test & Verification Plan

Tools to be used:

Category	Tool/Equipment	Purpose
Electrical Measurement	Digital Multimeter (DMM)	Voltage, current, and resistance measurements; continuity checks.
Electrical Measurement	Oscilloscope	Signal integrity analysis, PWM waveform verification, timing measurements.
Electrical Measurement	Logic Analyzer	Debugging I2C, UART, and other digital communication protocols; analyzing timing of digital signals.
Electrical Measurement	Bench Power Supply	Providing stable and adjustable power to the module during testing; simulating various power conditions.
Environmental Simulation	Environmental Chamber	Simulating varying temperature and humidity conditions for sensor calibration and stress testing.
Communication Analysis	USB-to-UART Converter	Interfacing the MCU's UART with a PC for data monitoring and injection.
Communication Analysis	Serial Port Terminal Software (e.g., PuTTY, Tera Term)	Displaying and logging UART communication; sending commands to the MCU.
Communication Analysis	Packet Sniffer/Protocol Analyzer (Software)	Analyzing custom UART packet structure, content, and checksum validity.

Sensor Testing:

In sensor testing, it is important to check both in a static environment and in a dynamic (changing/conditioned) environment.

For a Static environment:

First, need to put our temperature and humidity sensor into a controlled environment where the temperature and humidity are precisely known. This test will provide an understanding of how the sensor performs in a real environment.

After that using a high-precision calibrated thermometer and hygrometer placed next to the module's sensor we can Record readings from both the module's sensor and the reference instruments. This needs to take readings in regular intervals (e.g., every 5 seconds) for a longer period (e.g., 30 minutes) at different fixed temperature and humidity levels (e.g., 20°C/50%RH, 30°C/70%RH, 10°C/30%RH).

Then we can compare the module's sensor readings with the reference values. By Calculating the average error, standard deviation, and maximum deviation for both temperature and humidity we can verify that the readings are within the sensor's accuracy range.

Dynamic Environment:

In an environmental chamber where the temperature and humidity rapidly changing, we can check the values by using our sensors as well as our reference instruments (high-precision calibrated thermometer and hygrometer). We can set points (e.g., from 20°C to 35°C, then back to 20°C; from 40%RH to 80%RH, then back to 40%RH) for better analyzing.

After that we need to continuously log sensor readings from the module and reference instruments during these transitions.

By Plot the sensor readings against time for both the module and the reference we can Evaluate the sensor's response time (time to reach a stable reading after a change) and its ability to track dynamic changes without significant overshoot or undershoot. We need to be aware about any hysteresis effects.

EEPROM Verification

By following below steps, the integrity and persistence of data stored in the EEPROM can be thoroughly verified. That will ensure critical min/max readings are reliably maintained across power cycles and operational periods.

1. Initial EEPROM State Check

Before any write operations, read the entire content of the EEPROM to ensure it is either empty or contains expected default values. This can be done via a debugger or a dedicated EEPROM read utility.

If the EEPROM stores a checksum for its data, we can verify its validity to confirm data integrity from previous operations or factory defaults.

2. Min/Max Data Write and Read Verification

We can validate the min/max functionality by simulating controlled environmental changes. For example, we can begin with a low temperature and humidity, then gradually increase both to trigger new maximum values, and finally bring them back down to create new minimum values. This ensures that the full range of conditions is captured.

During these transitions, the MCU's internal min/max registers will be monitored in real time. Each time the stored values update, we can trigger a save to EEPROM (either manually or through the automatic save cycle). Immediately after each save, the values will be read back from EEPROM and compared with the expected values.

By following this process, we can confirm that the EEPROM correctly stores and retrieves min/max readings without data loss or corruption. Any mismatches between expected and stored values would highlight issues in the save or read-back process.

3. Power Cycle Persistence Test

To verify that the EEPROM correctly preserves data across power cycles, we can perform a repeatable sequence of tests. First, the system is allowed to run long enough to generate several min/max updates. Just before shutting down, the most recent min/max values are recorded for reference.

Next, the module (including the MCU and EEPROM) is fully powered down, ensuring that any residual charge is discharged. After a short pause, the system is powered back on. During initialization, the firmware will automatically retrieve the stored min/max values from EEPROM.

These retrieved values are then compared against the recorded values from before the shutdown. They should match exactly. By repeating this process across multiple cycles (e.g., 10–20 power cycles), we can confirm the robustness and reliability of EEPROM data retention under real-world operating conditions.

UART Packet Validation

To validate the UART communication and packet structure we can apply manual as well as automated procedures.

Considering a **more manual method**, we first establish a proper monitoring setup. A UART-to-USB converter is used to connect the MCU to a PC, allowing data transfer via the UART protocol. On the PC side, serial terminal software such as PuTTY, Tera Term, or RealTerm is configured with the correct UART settings (baud rate, data bits, parity, and stop bits) as defined by the MCU. The software is also configured to log all incoming data into a file for later inspection.

Once the monitoring environment is ready, the MCU is triggered to send known status packets—for example, predictable sensor readings, button states, or fan status updates. These transmissions are captured continuously for several minutes to generate a sufficient dataset.

The logged file is then reviewed manually to confirm that each packet follows the defined format (start byte, length, payload, checksum, end byte). The inspection also checks for correct byte ordering, consistent field lengths, and valid checksum values.

By following this procedure, we can verify both the integrity of the packet structure and the correctness of the transmitted data content.

Other than that method, more industrial friendly **Automated Verifications also can be used**. There an automated script can be used to parse, validate, and log UART data. The script identifies packet boundaries using the defined start and end delimiters, then extracts each field (such as sensor readings, button states, or fan status). Once decoded, the values are compared against the expected results. For example, if the MCU is configured to transmit a fixed temperature of 25.0 °C, the script verifies that the decoded packet correctly represents this value. Any deviations are flagged and included in a validation report, ensuring content accuracy without requiring manual checks.

In addition to field validation, the script performs checksum or CRC verification. The same checksum algorithm used in the MCU firmware (e.g., CRC-8, CRC-16, or XOR sum) is implemented in the parsing script. For each received packet, the script recalculates the checksum over the payload (excluding the checksum byte) and compares it with the checksum included in the packet. Any mismatch indicates either transmission errors or a discrepancy in checksum generation.

To confirm the robustness of this method, purposefully corrupted packets can also be introduced into the data stream. The script should consistently detect and report these errors, proving its effectiveness in identifying data corruption.

Input Buttons and Contact/Bump Switch Testing

To ensure reliable operation of the input buttons (Start, Stop, Reset) and contact/bump switches, several tests are carried out to verify functionality, debounce performance, response timing, and resilience against false triggers.

1) Basic Functionality and Interrupt Verification

- Each button and switch are activated individually, and the MCU's response is observed via UART messages to the host SBC. The expected control signal (e.g., "Start" or "Bump Detected") must be transmitted correctly.
- If LED indicators are present, they are checked for immediate visual feedback on activation.
- At the firmware level, ISRs (Interrupt Service Routines) are monitored using a JTAG/SWD debugger. Each press should trigger the ISR exactly once, confirming correct interrupt handling.

2) Debounce Effectiveness Testing

- **Rapid Activation:** Buttons and switches are pressed and released quickly in succession. The system should log only one logical event per press/release, with no duplicate triggers.
- **Signal Inspection:** Using an oscilloscope or logic analyzer, the raw input signal (showing contact bounce) and the debounced signal are compared. The debounced signal should generate only one clean transition.
- **Test Rig (Optional):** A dedicated debounce test rig may be used to simulate different bounce patterns, ensuring the firmware debounce algorithm effectively filters noise while preserving valid activations.

3) Response Timing Test

- The delay between a physical press/activation and the corresponding UART control signal is measured. An oscilloscope or logic analyzer is triggered on the input signal, and the time to the first UART packet bit is recorded.
- Multiple measurements are taken to confirm response consistency. Any significant jitter may indicate ISR delays or high CPU load.

4) False Trigger Prevention

- Noise Immunity: Electrical noise is introduced (e.g., nearby motors, injected signals) to test resilience. The system should not register unintended button/switch activations.
- Environmental Stress: Tests are repeated under different conditions such as temperature extremes and humidity, ensuring the components maintain stable performance without false triggers.

Data Logging Strategy

A robust logging system is essential to verify system reliability and behavior over extended test periods. For this purpose, a centralized logging approach on the host **Single Board Computer (SBC)** is recommended. The SBC collects data from the multi-sensor module via UART and records all relevant parameters for later analysis.

Logging Application

A simple Python script or lightweight application will continuously read data from the UART port, parse incoming packets, and extract key fields such as temperature, humidity, fan status, button/switch activations, and communication metrics.

Timestamping

Each record must be timestamped with millisecond precision at the moment it is received to ensure accurate sequencing of events.

Data Storage Formats

- **CSV:** Simple, human-readable, easily analyzed in spreadsheets.
- **JSON Lines (JSONL):** Flexible format suitable for structured or nested data.
- **SQLite Database:** Ideal for long-term testing and efficient querying.

Data Points to Log

- Sensor readings: temperature and humidity.
- Minimum/maximum values stored in EEPROM.
- Fan status (On/Off, PWM duty cycle).

- Button presses and bump switch activations (event-based logging).
- UART communication statistics (packets sent/received, checksum errors, malformed packets).
- System error codes or warnings from the MCU.

Logging Frequency

- Sensor data: every **1–5 seconds** for detailed trend analysis.
- Event data: logged immediately upon detection.
- Communication/error metrics: logged periodically (e.g., once per minute).

Post-Logging Analysis

After the test period, logs should be reviewed to:

- Identify long-term trends in sensor readings.
- Verify min/max values against EEPROM data.
- Correlate system responses with user actions (e.g., button presses).
- Calculate error rates in communication.
- Assess overall performance and responsiveness.

Archiving

All log files must be archived with clear, time-stamped filenames (e.g., 20250831_ModuleTest_Run1.csv) and stored in a backed-up location. This ensures traceability and supports future comparison across multiple test runs.

Section 2 – SBC Application & Integration Task

1. System Integration Diagram

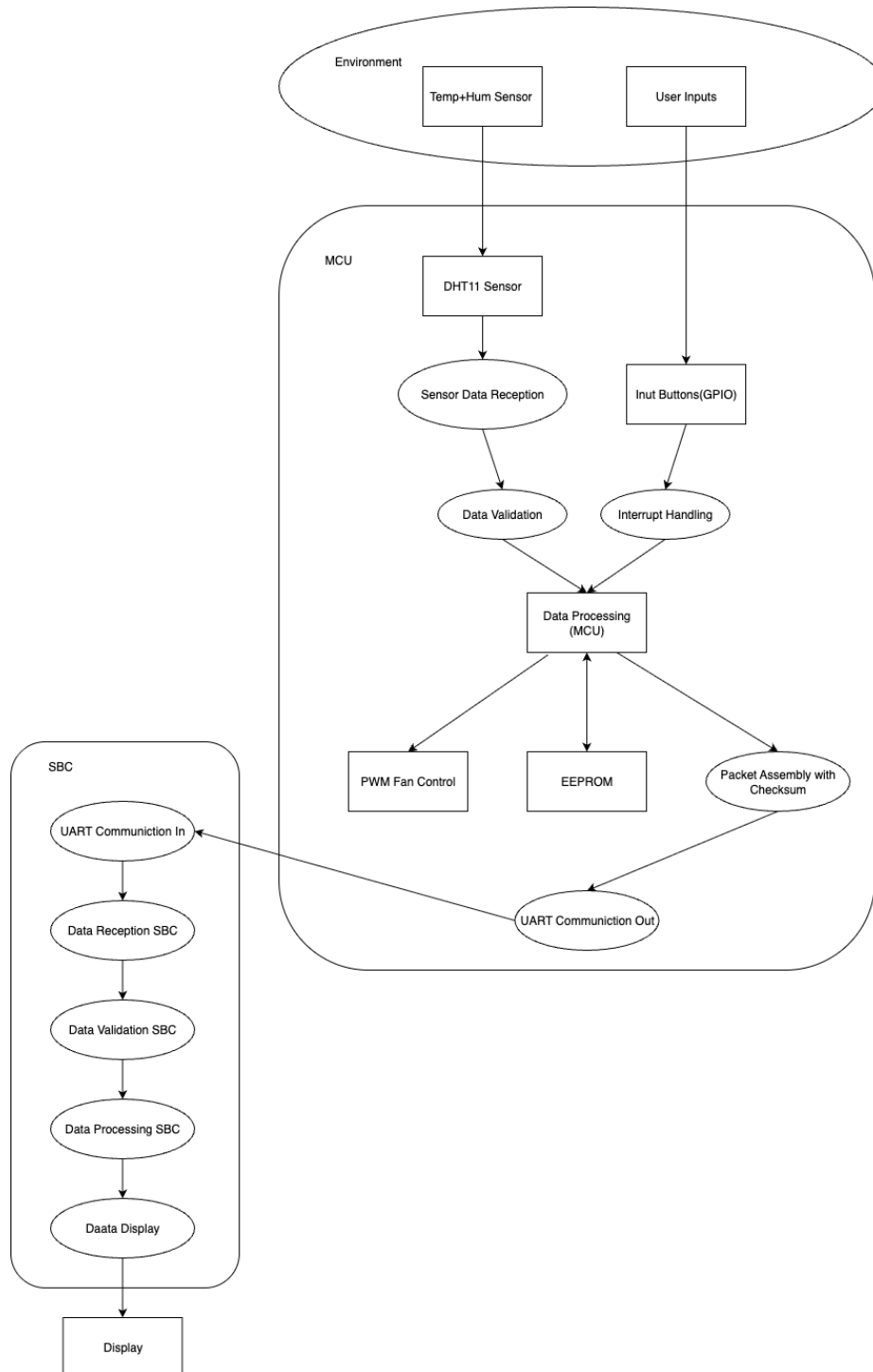


Figure 1 : System Integration Diagram

2. C++ Program Design & Implementation

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <string>
#include <vector>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <chrono>
#include <ctime>
#include <stdexcept>
#include <map>
#include <cstring>
#include <thread>

#include <sys/select.h>

/* CRC8 calculation (example, can be replaced with CRC16/32 if needed), Here i have replaced the
usual checksum checking method with more strong algorithm which is CRC-8. This could be use
for more industrial level rather just for the code so i have added that here*/

// Polynomial: 0x07 ( $x^8 + x^2 + x^1 + x^0$ ) , Basicly i generate CRC-8 representation
uint8_t calculateCrc8(const std::vector<uint8_t>& data) {
    uint8_t crc = 0x00; // Initial value
    uint8_t polynomial = 0x07; // CRC-8-ATM polynomial

    for (uint8_t byte : data) {
        crc ^= byte;
        for (int i = 0; i < 8; ++i) {
            if (crc & 0x80) {
                crc = (crc << 1) ^ polynomial;
            } else {
                crc <<= 1;
            }
        }
    }
    return crc;
}

// This Function is used to get current time-date for the logging purposes.
std::string getCurrentTimestamp() {
    auto now = std::chrono::system_clock::now();
    std::time_t current_time = std::chrono::system_clock::to_time_t(now);
    std::stringstream ss;
    ss << std::put_time(std::localtime(&current_time), "%Y-%m-%d %H:%M:%S");
    return ss.str();
}

/*Here im using this function to parse the data from the datapacket, assuming
data packet structure: TEMP:value,HUM:value,ACT:value where TEMP and HUM are
decimal numbers and ACT is an integer */
struct SensorData {
    double temperature;
    double humidity;
    int actuatorStatus;
    // Add more fields as needed

    std::string toString() const {
        std::stringstream ss;
        ss << "TEMP:" << std::fixed << std::setprecision(2) << temperature
            << ",HUM:" << std::fixed << std::setprecision(2) << humidity
```

```

        << ",ACT:" << actuatorStatus;
        return ss.str();
    }
};

// Function to parse the data part of the packet
SensorData parseSensorData(const std::string& dataPart) {
    SensorData data;
    std::map<std::string, std::string> parsedFields;

    size_t start = 0;
    size_t end = dataPart.find(",");
    while (end != std::string::npos) {
        std::string segment = dataPart.substr(start, end - start);
        size_t colonPos = segment.find(":");
        if (colonPos != std::string::npos) {
            std::string key = segment.substr(0, colonPos);
            std::string value = segment.substr(colonPos + 1);
            parsedFields[key] = value;
        }
        start = end + 1;
        end = dataPart.find(",", start);
    }
    // Handle the last segment
    std::string segment = dataPart.substr(start);
    size_t colonPos = segment.find(":");
    if (colonPos != std::string::npos) {
        std::string key = segment.substr(0, colonPos);
        std::string value = segment.substr(colonPos + 1);
        parsedFields[key] = value;
    }

    try {
        data.temperature = std::stod(parsedFields["TEMP"]);
        data.humidity = std::stod(parsedFields["HUM"]);
        data.actuatorStatus = std::stoi(parsedFields["ACT"]);
    } catch (const std::out_of_range& oor) {
        throw std::runtime_error("Data parsing error: Value out of range");
    } catch (const std::invalid_argument& ia) {
        throw std::runtime_error("Data parsing error: Invalid argument");
    }

    return data;
}

// Function to configure serial port for the transmission and reception of data
int configureSerialPort(const std::string& port_name, int baud_rate) {
    int serial_fd = open(port_name.c_str(), O_RDWR | O_NOCTTY | O_NONBLOCK); // Use O_NONBLOCK for
select()
    if (serial_fd == -1) {
        std::cerr << "Error: Cannot open serial port " << port_name << ": " << strerror(errno) <<
std::endl;
        return -1;
    }

    struct termios tty;
    if (tcgetattr(serial_fd, &tty) != 0) {
        std::cerr << "Error from tcgetattr: " << strerror(errno) << std::endl;
        close(serial_fd);
        return -1;
    }

    cfsetospeed(&tty, baud_rate);
    cfsetispeed(&tty, baud_rate);

    tty.c_cflag |= (CLOCAL | CREAD); // Enable the receiver and set local mode
    tty.c_cflag &= ~PARENB; // No parity
    tty.c_cflag &= ~CSTOPB; // 1 stop bit
    tty.c_cflag &= ~CSIZE; // Clear all the size bits

```

```

tty.c_cflag |= CS8;                // 8 data bits
tty.c_cflag &= ~CRTSCTS;          // Disable RTS/CTS hardware flow control

tty.c_lflag &= ~ICANON;           // Disable canonical mode (raw input)
tty.c_lflag &= ~ECHO;             // Disable echo
tty.c_lflag &= ~ECHOE;            // Disable erasure
tty.c_lflag &= ~ECHONL;          // Disable new-line echo
tty.c_lflag &= ~ISIG;             // Disable interpretation of INTR, QUIT and SUSP

tty.c_iflag &= ~(IXON | IXOFF | IXANY); // Turn off s/w flow ctrl
tty.c_iflag &= ~(IGNBRK|BRKINT|PARMRK|ISTRIP|INLCR|IGNCR|ICRNL); // Disable any special handling of
received bytes

tty.c_oflag &= ~OPOST;            // Prevent special interpretation of output bytes (e.g. newline
chars)
tty.c_oflag &= ~ONLCR;            // Prevent conversion of newline to carriage return/line feed

tty.cc[VTIME] = 0;                // Wait for 0.0 seconds (non-blocking)
tty.cc[VMIN] = 0;                // Read 0 characters (non-blocking)

if (tcsetattr(serial_fd, TCSANOW, &tty) != 0) {
    std::cerr << "Error from tcsetattr: " << strerror(errno) << std::endl;
    close(serial_fd);
    return -1;
}

return serial_fd;
}

// Global log file stream
std::ofstream g_logFile;

// Function to open/reopen log file with rotation logic
void openLogFile(const std::string& log_path, long max_size_mb) {
    if (g_logFile.is_open()) {
        g_logFile.close();
    }

    // Check current log file size for rotation
    std::ifstream check_file(log_path, std::ios::ate | std::ios::binary);
    if (check_file.is_open()) {
        long current_size = check_file.tellg();
        check_file.close();
        if (current_size > max_size_mb * 1024 * 1024) {
            std::string old_log_path = log_path + ".old";
            std::rename(log_path.c_str(), old_log_path.c_str());
            std::cerr << getCurrentTimestamp() << " - Log file rotated to " << old_log_path << std::endl;
        }
    }

    g_logFile.open(log_path, std::ios::app);
    if (!g_logFile.is_open()) {
        std::cerr << getCurrentTimestamp() << " - Error: Cannot open log file " << log_path << std::endl;
    }
}

int main(int argc, char* argv[]) {
    std::string serial_port = "/dev/ttyUSB0";
    int baud_rate = B9600;
    std::string log_file_path = "sensor_data.log";
    long max_log_size_mb = 10; // 10 MB

    // Basic argument parsing
    for (int i = 1; i < argc; ++i) {
        std::string arg = argv[i];
        if (arg == "--port" && i + 1 < argc) {
            serial_port = argv[++i];
        } else if (arg == "--baud" && i + 1 < argc) {
            baud_rate = std::stoi(argv[++i]);
        }
    }
}

```

```

    } else if (arg == "--log" && i + 1 < argc) {
        log_file_path = argv[++i];
    } else if (arg == "--max-log-size" && i + 1 < argc) {
        max_log_size_mb = std::stol(argv[++i]);
    } else if (arg == "--help") {
        std::cout << "Usage: " << argv[0] << " [--port <serial_port>] [--baud <baud_rate>] [--log
<log_file_path>] [--max-log-size <MB>]\n";
        return 0;
    }
}

int serial_fd = configureSerialPort(serial_port, baud_rate);
if (serial_fd == -1) {
    return 1;
}

openLogFile(log_file_path, max_log_size_mb);

std::string packet_buffer;
const std::string PACKET_START_DELIMITER = "<PKT>";
const std::string PACKET_END_DELIMITER = "</PKT>";

while (true) {
    fd_set read_fds;
    FD_ZERO(&read_fds);
    FD_SET(serial_fd, &read_fds);

    struct timeval tv;
    tv.tv_sec = 1; // Timeout after 1 second
    tv.tv_usec = 0;

    int retval = select(serial_fd + 1, &read_fds, NULL, NULL, &tv);

    if (retval == -1) {
        std::cerr << getCurrentTimestamp() << " - Error in select(): " << strerror(errno) <<
std::endl;
        // Attempt to re-open serial port on error
        close(serial_fd);
        serial_fd = configureSerialPort(serial_port, baud_rate);
        if (serial_fd == -1) {
            // If re-opening fails, wait and try again
            std::this_thread::sleep_for(std::chrono::seconds(5));
            continue;
        }
    } else if (retval) {
        // Data is available
        char buffer[256];
        int n = read(serial_fd, buffer, sizeof(buffer));

        if (n > 0) {
            packet_buffer.append(buffer, n);

            size_t start_pos = packet_buffer.find(PACKET_START_DELIMITER);
            size_t end_pos = packet_buffer.find(PACKET_END_DELIMITER, start_pos != std::string::npos ?
start_pos + PACKET_START_DELIMITER.length() : 0);

            while (start_pos != std::string::npos && end_pos != std::string::npos) {
                std::string full_packet_str = packet_buffer.substr(start_pos +
PACKET_START_DELIMITER.length(), end_pos - (start_pos + PACKET_START_DELIMITER.length()));

                // Remove the processed packet from the buffer
                packet_buffer.erase(0, end_pos + PACKET_END_DELIMITER.length());

                // Expected format: "DATA:TEMP:25,HUM:60,ACT:1,CRC:XX"
                size_t crc_pos = full_packet_str.find(",CRC:");
                if (crc_pos != std::string::npos) {
                    std::string data_part_str = full_packet_str.substr(0, crc_pos);
                    std::string crc_str = full_packet_str.substr(crc_pos + 5);

```

```

        try {
            uint8_t received_crc = static_cast<uint8_t>(std::stoi(crc_str));

            // Convert data_part_str to vector<uint8_t> for CRC calculation
            std::vector<uint8_t> data_bytes(data_part_str.begin(), data_part_str.end());
            uint8_t calculated_crc = calculateCrc8(data_bytes);

            if (received_crc == calculated_crc) {
                try {
                    SensorData sensor_data = parseSensorData(data_part_str);
                    std::string log_entry = getCurrentTimestamp() + " - Valid Data: " +
sensor_data.toString();

                    std::cout << log_entry << std::endl;
                    if (g_logFile.is_open()) {
                        g_logFile << log_entry << std::endl;
                        openLogFile(log_file_path, max_log_size_mb); // Check for rotation
after writing
                    }
                } catch (const std::runtime_error& e) {
                    std::string error_log_entry = getCurrentTimestamp() + " - Data Parsing
Error: " + e.what() + " - Packet: " + full_packet_str;
                    std::cerr << error_log_entry << std::endl;
                    if (g_logFile.is_open()) {
                        g_logFile << error_log_entry << std::endl;
                    }
                }
            } else {
                std::string error_log_entry = getCurrentTimestamp() + " - Checksum
Mismatch: Received CRC=" + std::to_string(received_crc) + ", Calculated CRC=" +
std::to_string(calculated_crc) + " - Packet: " + full_packet_str;
                std::cerr << error_log_entry << std::endl;
                if (g_logFile.is_open()) {
                    g_logFile << error_log_entry << std::endl;
                }
            }
        } catch (const std::invalid_argument& ia) {
            std::string error_log_entry = getCurrentTimestamp() + " - CRC Parsing Error:
Invalid CRC value - Packet: " + full_packet_str;
            std::cerr << error_log_entry << std::endl;
            if (g_logFile.is_open()) {
                g_logFile << error_log_entry << std::endl;
            }
        } catch (const std::out_of_range& oor) {
            std::string error_log_entry = getCurrentTimestamp() + " - CRC Parsing Error:
CRC value out of range - Packet: " + full_packet_str;
            std::cerr << error_log_entry << std::endl;
            if (g_logFile.is_open()) {
                g_logFile << error_log_entry << std::endl;
            }
        }
    } else {
        std::string error_log_entry = getCurrentTimestamp() + " - Malformed Packet:
Missing CRC - Packet: " + full_packet_str;
        std::cerr << error_log_entry << std::endl;
        if (g_logFile.is_open()) {
            g_logFile << error_log_entry << std::endl;
        }
    }

    // Look for the next packet in the buffer
    start_pos = packet_buffer.find(PACKET_START_DELIMITER);
    end_pos = packet_buffer.find(PACKET_END_DELIMITER, start_pos != std::string::npos ?
start_pos + PACKET_START_DELIMITER.length() : 0);
}
} else if (n == 0) {
    // No data read, but select indicated data was available. This can happen with non-
blocking reads.
    // Or, the serial port might have been disconnected.

```

```

        std::cerr << getCurrentTimestamp() << " - Warning: Read returned 0 bytes. Serial port
might be disconnected or no data available.\n";
        // Consider re-opening the port or waiting for a bit
        close(serial_fd);
        serial_fd = configureSerialPort(serial_port, baud_rate);
        if (serial_fd == -1) {
            std::this_thread::sleep_for(std::chrono::seconds(5));
        }
    } else { // n < 0, error
        std::cerr << getCurrentTimestamp() << " - Error reading from serial port: " <<
strerror(errno) << std::endl;
        // Attempt to re-open serial port on error
        close(serial_fd);
        serial_fd = configureSerialPort(serial_port, baud_rate);
        if (serial_fd == -1) {
            // If re-opening fails, wait and try again
            std::this_thread::sleep_for(std::chrono::seconds(5));
            continue;
        }
    }
} else {
    // Timeout, no data within 1 second. Continue loop.
    // std::cout << getCurrentTimestamp() << " - No data received in 1 second.\n";
}
}

close(serial_fd);
if (g_logFile.is_open()) {
    g_logFile.close();
}
return 0;
}

```

3. Data Handling & Logging Plan

To ensure reliable operation, all incoming sensor data from the MCU will be logged in a structured format for later analysis. Each entry will include a timestamp, log level (e.g., valid data, warning, error), and message details. Valid data entries will contain parsed sensor readings, while errors and warnings will record issues like checksum mismatches, malformed packets, or communication failures. Log rotation will be used to prevent files from growing too large, and logs will be stored in a dedicated directory for easier management.

The system is designed for extensibility. New sensor fields (e.g., pressure) can be added by updating the MCU firmware to include the extra key-value pairs in the UART packets and making minimal changes on the SBC side. Since parsing is based on a flexible key-value approach, additional fields can be integrated with minimal disruption.

For corrupted or missing data, the system uses CRC checks to discard invalid packets and logs the errors with detailed context for debugging. Malformed packets are detected through strict delimiter checks, and parsing errors are safely caught without crashing the program. In case of missing data (e.g., serial

port disconnection), the system attempts automatic recovery and logs the issue. A watchdog or timeout mechanism can also be added to trigger alerts if data is not received for a defined period.

This approach ensures accurate logging, easy future expansion, and robust handling of errors or communication issues.

4. Linux Startup & Automation

Here I'm explaining the method I used to implement more human-less intervention for the program running. The `sbc_data_receiver.cpp` program will be configured to automatically start on a Linux-based Single Board Computer (SBC) upon system boot by using `'systemd'`.

`'systemd'` is the modern `'init'` system and service manager for Linux distributions, including those commonly used on SBCs. It provides a robust and flexible framework for managing services, handling dependencies, and ensuring processes are started and maintained correctly. I will create a `systemd` service unit file to manage the `sbc_data_receiver` application.

`'systemd'` service unit file defines how a service should be managed. It typically resides in path `'/etc/systemd/system/'`.

```
[Unit]
```

```
Description=SBC Sensor Data Receiver Service
```

```
After=network.target serial-getty@ttyUSB0.service
```

```
[Service]
```

```
Type=simple ExecStart=/usr/local/bin/sbc_data_receiver --port  
/dev/ttyUSB0 --baud 115200 --log  
/var/log/sbc_sensor_data/sensor_data.log --max-log-size 50 Restart=on-  
failure RestartSec=5s User=sbcuser Group=sbcuser  
WorkingDirectory=/var/lib/sbc_data_receiver StandardOutput=journal  
StandardError=journal
```

```
[Install]
```

```
WantedBy=multi-user.target
```

[Unit] : It starts after the network is up, and care is taken to avoid conflicts with `serial-getty@ttyUSB0.service` since the serial port is used for data, not login.

[Service] :

- Runs the `sbc_data_receiver` binary on `/dev/ttyUSB0` at 115200 baud, logging to `/var/log/sbc_sensor_data/`.
- Configured with `Restart=on-failure` and a 5-second delay, so it will automatically recover from crashes.
- Runs as an unprivileged `sbcuser` with proper group permissions for security.
- Uses `/var/lib/sbc_data_receiver` as its working directory, and sends output to the 'systemd' journal for easy log viewing.

[Install] : Enabled under `multi-user.target`, so it comes up automatically on boot in a normal multi-user (non-GUI) environment.

Running After Unexpected Reboots

The service is designed to come back online automatically if the SBC reboots, such as after a power outage. Because of `Restart=on-failure` and `WantedBy=multi-user.target`, 'systemd' will bring the program back up as soon as the system is ready. The `After=` directive ensures dependencies like the serial port are available first.

Logging and Error Handling at Startup

All output from the program (*stdout and stderr*) is captured by the 'systemd' journal, making it easy to check startup messages and errors using `journalctl`. At the same time, the program writes its own detailed sensor data and error logs to a dedicated log file with rotation, preventing uncontrolled growth. This combination gives both system-level and application-level visibility.

Minimal Manual Intervention

The system is built to be "set and forget":

- Services restart automatically if they fail.
- They start on boot without manual action.
- Logs are centralized in `journalctl` for quick inspection.
- Program parameters (port, baud rate, log location, etc.) can be adjusted at runtime without rebuilding.

Remote Management:

For long-term or industrial deployments, remote management strengthens reliability:

- SSH enables checking service status, viewing logs, and restarting services without physical access.
- Monitoring tools (Prometheus, Nagios, etc.) can provide alerts on failures or performance issues.
- OTA updates make it possible to roll out software or firmware upgrades without onsite work.

Together, these measures ensure the data receiver runs reliably with little to no manual intervention, even in production environments.

5. Test & Verification Plan

In this section the goal of this plan is to confirm that the `sbc_data_receiver` program can reliably handle MCU sensor data while gracefully dealing with errors, disconnections, and high-load conditions.

5.1. Simulating Incoming Data

Since real hardware (ESP32 and sensors) may not always be available, we'll use simulation methods to generate repeatable test scenarios.

Python Serial Port Simulator

A Python script can be used to emulate the ESP32, sending UART packets in the expected format. This allows full control over data content, timing, and error conditions without relying on physical hardware.

Key Features:

- Configurable sensor data (temperature, humidity, actuator status).
- Proper framing and CRC-8 checksums for valid packets.
- Error injection options: invalid CRC, malformed packets, corrupted or missing data, and burst traffic.
- Support for both real and virtual serial devices.

Virtual Serial Ports

For development or CI testing without hardware, we can create linked pseudo-terminals using `socat`. Data written to one end is read from the other, effectively simulating a serial cable.

Example setup:

```
sudo apt-get install socat  
  
socat -d -d pty,raw,echo=0 pty,raw,echo=0
```

This provides two device paths (e.g., `/dev/pts/0` and `/dev/pts/1`), where one can be connected to the simulator and the other to `sbc_data_receiver`.

5.2. Verify Correct Parsing and Data Integrity Checks

This section lays out how to verify that `sbc_data_receiver` correctly parses incoming packets, enforces CRC checks, and behaves robustly under errors and load.

Unit tests (recommended)

Write fast, isolated tests for the pure functions:

- **`calculateCrc8`** — verify CRC outputs against known inputs (use a reference implementation/online calculator).
- **`parseSensorData`** — feed a range of `dataPart` strings and assert the returned `SensorData` fields, including edge cases (empty fields, negative values, type limits).

These unit tests are cheap, deterministic, and catch logic bugs early.

Integration testing with the serial simulator

Run the `sbc_data_receiver` alongside a Python serial simulator (or a virtual serial pair) so you can exercise real I/O without physical hardware. Check receiver output, logs, and program stability while the simulator injects different scenarios.

Key test cases to run

1. **Valid data flow**
 - Send many valid packets with varied sensor values.
 - Expect: all logged as valid, stdout display matches sent values, parsed fields correct.
2. **Checksum mismatch detection**
 - Send packets with wrong CRC.
 - Expect: `Checksum Mismatch` errors logged; corrupted packets discarded (not shown as valid).
3. **Malformed packet handling**
 - Send missing delimiters, incomplete or invalid key/value pairs.

- Expect: `Malformed Packet` or `Data Parsing Error` logged; no crashes, no bad data accepted.
4. Boundary conditions
 - Test min/max sensor values and other edge cases.
 - Expect: correct parsing and display, no overflow or truncation errors.
 5. Performance / stress testing
 - Blast high-rate traffic to evaluate throughput and latency.
 - Monitor for dropped packets, lag, and CPU/memory spikes.

How to verify results

- Log-file analysis: script the parsing of `sensor_data.log` to count valid packets, checksum failures, and parsing errors; validate content against what the simulator sent.
- Journal / console: watch `stdout/stderr` in real time (or via `journalctl`) for immediate errors.
- Debugging: use `gdb` or run a debug build to step through failure scenarios when needed.

3. Test Recovery from Disconnections & Communication Errors

Since industrial deployments must withstand unstable connections, these tests ensure that `sbc_data_receiver` can detect failures, recover gracefully, and resume normal operation without corrupting data.

How to simulate disconnections

- Physical disconnect: Unplug/replug the UART cable or USB-to-serial adapter.
- Virtual disconnect: If using `socat`, kill the process or close one end of the virtual port.
- MCU reboot: Power-cycle the ESP32 to mimic an unexpected restart.

Test cases

1. Transient disconnection (brief unplug/replug)
 - Expect:
 - Error logged on disconnect.
 - Receiver retries port access.
 - Data flow resumes automatically after reconnect, without corruption.
2. Prolonged disconnection
 - Expect:
 - Receiver keeps retrying to open the port (e.g., every 5s).
 - CPU/memory usage stays stable during downtime.
 - Communication resumes immediately when port becomes available again.
3. SBC reboot during disconnection

- Expect:
 - Systemd starts the service automatically after reboot.
 - Receiver tries to reconnect until MCU/port are back online.
 - Data flow resumes seamlessly.
- 4. Baud rate mismatch
 - Expect:
 - Garbled data is logged as parsing/CRC errors.
 - Receiver continues running without crashing.

Section 3 – System Design Challenge

This document consists of hardware selection, system architecture, ROS2-related information, physical robot design sketches, AI models and information that can be used here, OTA update and monitoring flow, and ROS2 integration information.

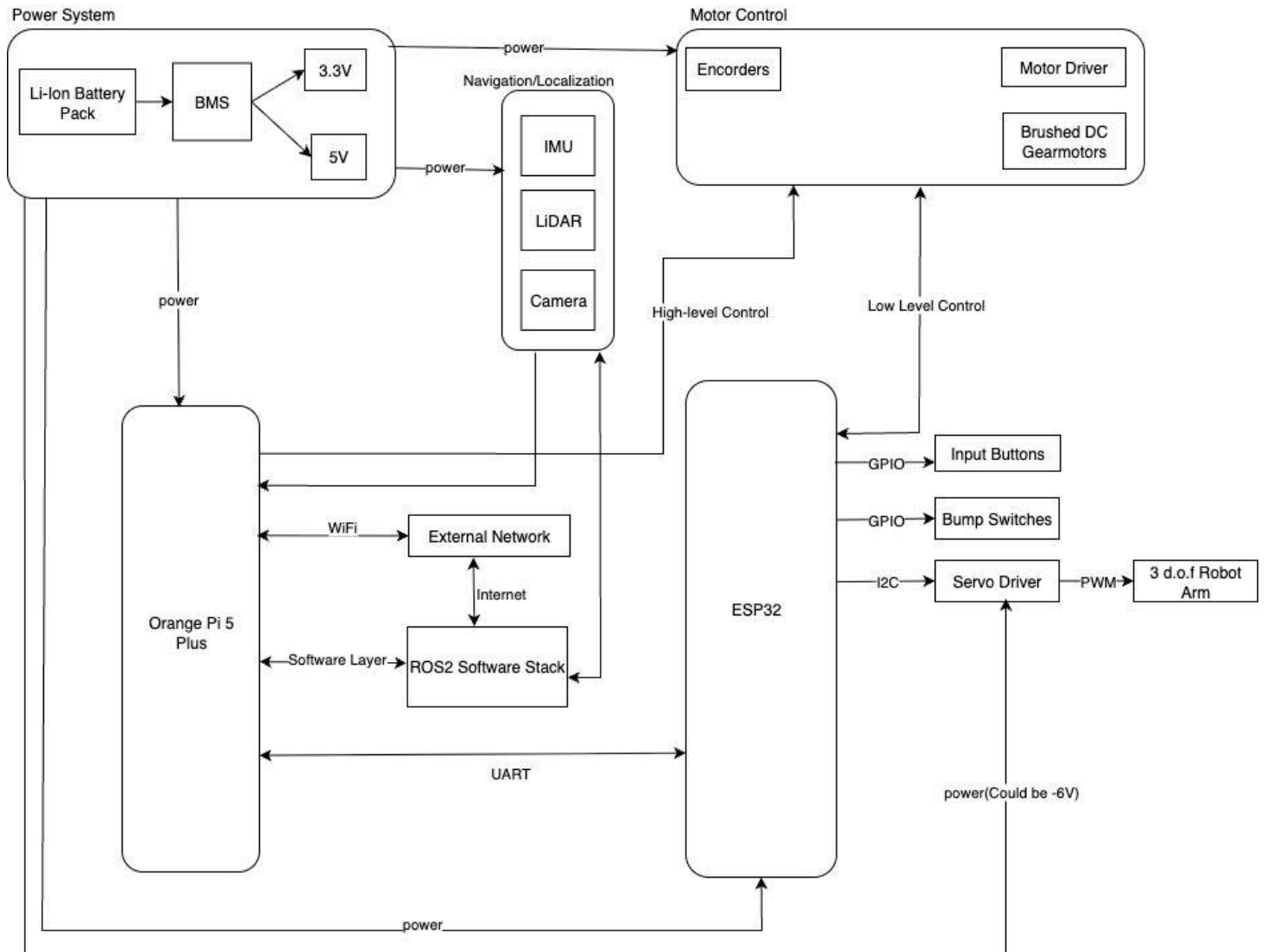
Key Requirements Analysis

- **Single-Board Computer (SBC) Operation:** As instructed in the assessment, we will use an SBC and an MCU, with the SBC acting as the main control unit.
- **Navigation and Localization:** As instructed, we are using LiDAR and IMU for navigation and localization in this robot project.
- **QR Code Recognition:** Since QR code recognition is a required feature, we are incorporating a camera module for this purpose.
- **OTA Firmware Updates:** Over-The-Air (OTA) updates are another expected feature, integrated within both the SBC and MCU for improved maintainability.
- **Cost-Effectiveness:** This criterion is a primary consideration in our architectural design and component selection.
- **Indoor Operation:** This is another requirement and is suitable for the design.
- **Package Delivery:** A robot arm has been added to handle this part fully autonomously, as instructed in the assessment.

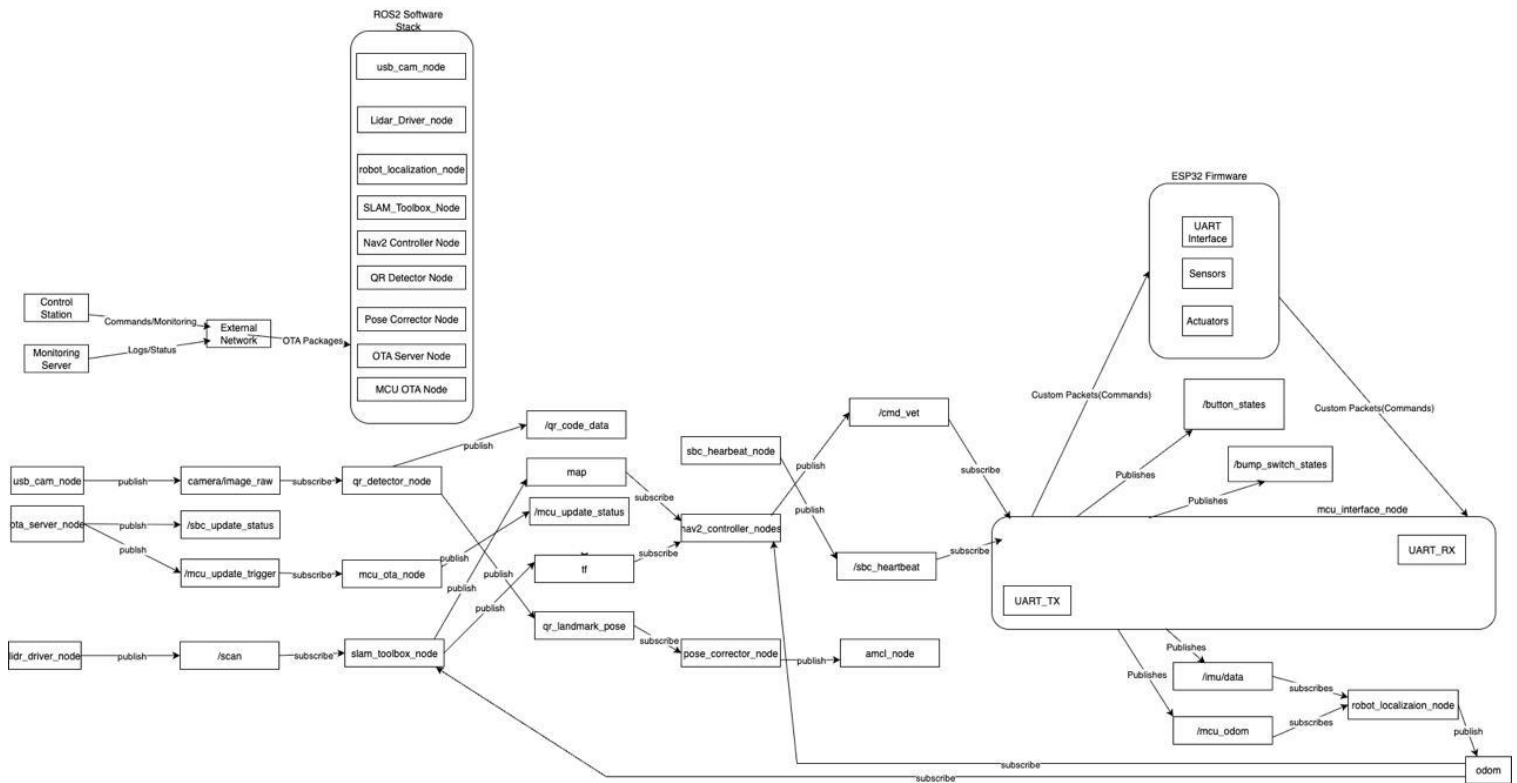
Category	Component	Specific Model/Type	Justification	Cost/ Effectiveness	Integration	Performanc e
Onboard Computer (SBC)	Single-Board Computer	Orange Pi 5 Plus	High performance-to-cost ratio, powerful CPU/GPU for ROS2, ample RAM, rich I/O	High	Excellent (Ubuntu/ROS2)	High
Motors	Brushed DC Gearmotors with Encoders	Pololu 25D mm Metal Gearmotors (e.g., 30:1 or 50:1 gear ratio)	Cost-effective, good torque/speed balance, integrated encoders for odometry	High	Easy (ESP32 PWM/GPIO)	Medium-High
Motor Controllers	Dual-Channel Brushed DC Motor Driver	Based on DRV8835/DRV8871 (low current) or BTS7960B (high current)	Bidirectional control, PWM compatible, interfaces with ESP32	High	Easy (ESP32 PWM/GPIO)	Medium
Navigation Sensor (LiDAR)	2D LiDAR	RPLIDAR A1M8 or C1	Cost-effective, 360-degree scan, sufficient range for indoor SLAM.	High	Easy (USB to SBC, ROS2 packages)	Medium
Navigation Sensor (IMU)	Inertial Measurement Unit	MPU-6050 (6-axis) or MPU-9250/9265 (9-axis)	Affordable, provides orientation/acceleration data for sensor fusion and odometry correction	High	Easy (I2C to ESP32)	Medium
Perception (Camera)	USB Webcam	Logitech C920 (or similar Full HD USB Webcam)	Cost-effective, sufficient resolution for QR code detection, plug-and-play.	High	Easy (USB to SBC, usb_cam ROS2)	Medium
Internal Communication	UART / Serial	Direct connection between Orange Pi 5 Plus and ESP32	Simple, low-latency, reliable for SBC-MCU data exchange.	High	Direct wiring	High
External Communication	Wi-Fi	Integrated on Orange Pi 5 Plus	Ubiquitous, sufficient bandwidth for remote control/monitoring/OTA	High	Integrated	High
Battery	Li-ion Battery Pack	18650 Li-ion cells (e.g., 3S1P or 3S2P)	High energy density, cost-effective, common in robotics.	High	Requires BMS	High
Power Management	Buck Converters & BMS	LM2596/MP1584 (5V), AMS1117-3.3 (3.3V), 3S BMS	Efficient voltage regulation, battery protection	High	Standard circuitry	High
Bump Switches	Contact Switches	Lever-actuated Micro Switches	Physical obstacle detection for safety	High	Easy (GPIO to ESP32)	High
Input Buttons	Tactile Push Buttons	Standard Momentary Buttons	Simple user input for Start/Stop/Reset	High	Easy (GPIO to ESP32)	High

Package Handling	3-DOF Robot Arm	3x MG996R, 1x SG90 + PCA9685	Flexible, cost-effective, precise enough for packages	Medium	Easy (I2C to ESP32)	Medium
------------------	--------------------	---------------------------------	---	--------	------------------------	--------

2. High Level System Architecture



3. ROS2 Node Graph



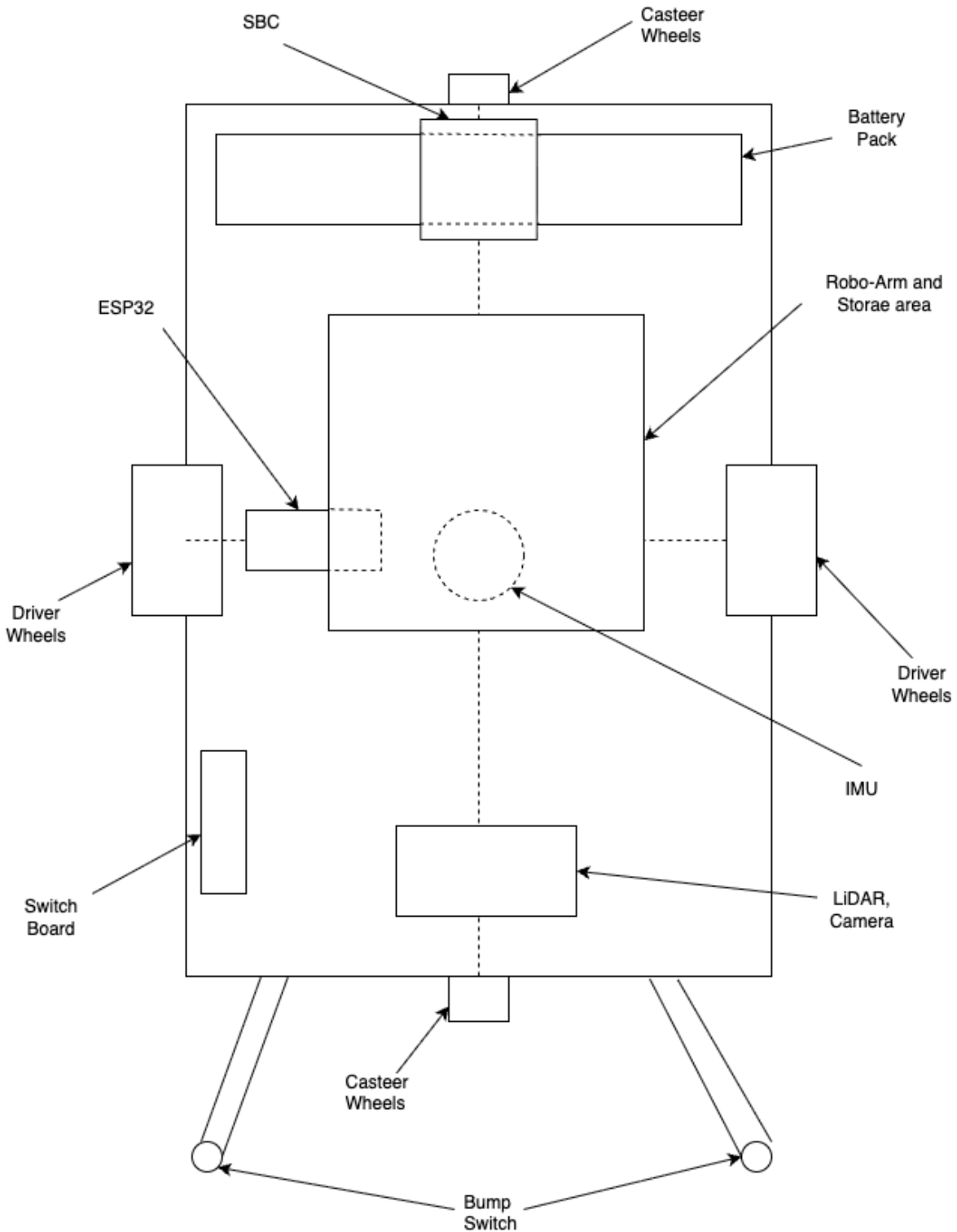
This graph is maybe not clear, you can visit this link to revise the draw.io link where I have made the original node-graph.

link -

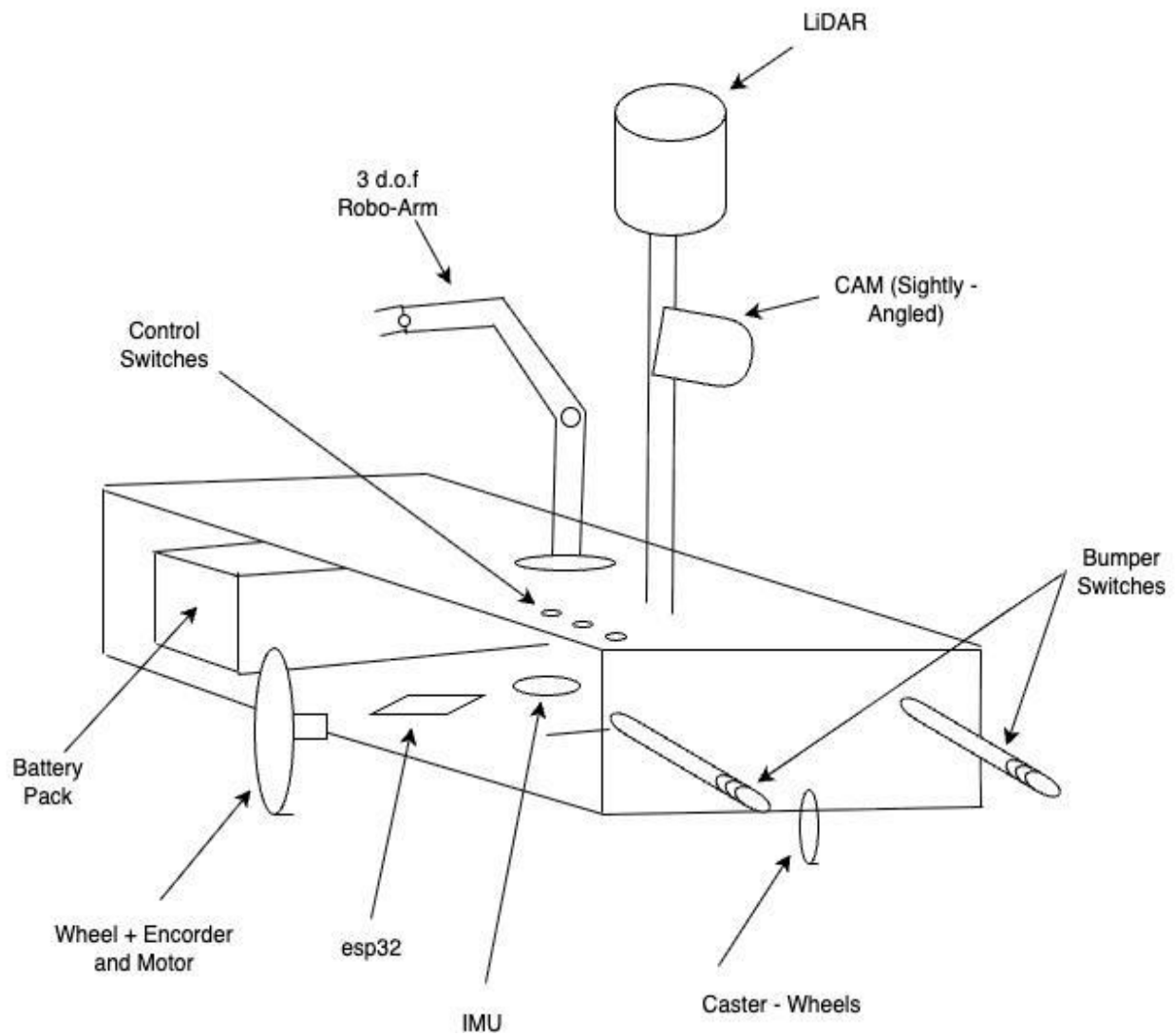
<https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&target=blank&highlight=0000ff&edit=blank&layers=1&nav=1&dark=auto#G15aNFpWp9jIW0FC340rVHFnPgPDgMdtYo>

4. Physical Robot Design Sketch

Plane View:



Side View:



Main Reference Frame

1. **base_link** frame: This is the main physical center of the frame. All other component frames, including those for sensors and actuators, will be defined as fixed transforms relative to the **base_link**. Therefore, I have used the physical center of the main frame as the **base_link** frame and positioned the IMU there, as shown in the physical design.
2. **odom** frame: The **odom** frame represents the robot's local reference frame for motion tracking. It defines the robot's pose relative to its starting position. In our design, odometry is computed by fusing wheel encoder data from the MCU (ESP32) with inertial measurements from the IMU, using the ROS 2 **robot_localization** package.
3. **map** frame: The **map** frame is the global reference frame for the robot's operating environment. It represents a fixed, world-level coordinate system generated by a Simultaneous Localization and Mapping (SLAM) system. When considering our design, the 2DLiDAR (**lidar_driver_node** + **slam_toolbox/amcl_node**) maintains the map. QR code detections can act as landmarks for pose correction, helping to realign the robot in the **map** frame.

After the main frames, each sensor and actuator have its own dedicated coordinate frame. I define these frames as static transforms to the **base_link** frame, meaning their positions and orientations do not change with respect to the robot's body.

4. **lidar_link**: The LiDAR is mounted at the top of a front-facing pole. The **lidar_link** frame's origin is at the center of the LiDAR's scanning plane. Relative to **base_link**, this frame has a forward x offset (since it's at the front of the chassis) and a positive z offset (height of the pole). Its roll, pitch, and yaw remain zero, as the LiDAR is aligned flat with the ground.
5. **imu_link**: The IMU is mounted at the center of the chassis, exactly at the **base_link** origin. Therefore, the **imu_link** frame is coincident with the **base_link**. No x, y, or z offsets are required, and the IMU axes are aligned with the robot's axes, simplifying sensor fusion with wheel odometry.
6. **camera_link**: The USB camera is mounted on the same front pole, positioned just below the LiDAR. The **camera_link** frame's origin is at the optical center of the camera lens. Its transform from **base_link** includes the same forward x offset as the LiDAR, but a slightly smaller z offset (below the LiDAR). Additionally, its orientation includes a downward pitch angle to ensure it captures QR codes and landmarks on walls or drop-off points.
7. **left_wheel_link**: This frame represents the left drive wheel, with its origin located at the center of the wheel's axle. Relative to the **base_link**, it is positioned with a negative y offset, placing it on the left side of the chassis, and has a zero or minimal z offset since the wheel contacts the ground plane. The y-axis, following the REP-103 standard, defines the wheel's rotation axis and thus its spin direction.
8. **right_wheel_link**: This frame represents the right drive wheel, also with its origin at the wheel's axle center. Relative to the **base_link**, it is positioned with a positive y offset, placing it on the

right side of the chassis, and has zero or minimal z offset. Its rotation axis mirrors that of the left wheel, and together, the two wheels enable differential drive motion.

Ensuring TF Tree Accuracy Over Time

Following strategies will be employed to ensure the TF tree remains accurate throughout the robot's operation:

1. **Accurate URDF/XACRO Definition:** The initial and most critical step involves precisely measuring and defining all static transforms within the robot's URDF or XACRO file. Any measurement errors or inaccuracies introduced at this stage will propagate throughout the entire system. Therefore, meticulous attention to detail during the physical construction and subsequent digital modeling is essential.
2. **Calibration:** Sensors are subject to various biases and inaccuracies that can drift over time. Regular and thorough calibration procedures are necessary to ensure their data is accurate and properly aligned with the robot's coordinate system. Key calibration efforts include:
 - i) **IMU Calibration:** Calibrating the accelerometer, gyroscope, and magnetometer to correct for biases, scale factors, and misalignments.
 - ii) **LiDAR-IMU Extrinsic Calibration:** If sensor fusion techniques are employed (e.g., for state estimation), the extrinsic calibration (i.e., the precise relative pose) between the LiDAR and IMU must be highly accurate.
 - iii) **Camera Calibration:** Calibrating the camera involves determining its intrinsic parameters (such as focal length, principal point, and distortion coefficients) and its extrinsic parameters (its pose relative to base_link).
 - iv) **Robotic Arm Calibration:** For the 3 DoF robotic arm, calibration of its joint limits, home positions, and potentially its end-effector pose is vital.
3. **Robust Odometry Source:** The odom -> base_link transform, which represents the robot's local motion, will be published by a robust odometry source. Ideally, this involves fusing data from multiple sensors, primarily wheel encoders and the IMU.
4. **Global Localization (SLAM/AMCL):** The map -> odom transform, which corrects for accumulated drift in the odometry, will be provided by a global localization algorithm. This typically involves a SLAM (Simultaneous Localization and Mapping) system that builds a map while simultaneously localizing the robot within it, or a localization-only approach like Adaptive Monte Carlo Localization (AMCL) that estimates the robot's pose within a pre-built map. These algorithms continuously refine the robot's position and orientation within the global map frame, ensuring long-term accuracy.
5. **Monitoring TF Tree with rqt_tf_tree:** During development, testing, and even deployment, regularly monitoring the TF tree using tools like rqt_tf_tree is invaluable. It helps in quickly

identifying common issues such as disconnected frames, incorrect parent-child assignments, or frames that are not being published, which are critical for debugging and maintaining accuracy.

5. AI Model(s) & Runtime

The main task we can use AI models here in this robot is QR code detection for identifying delivery locations. While deep learning models can be applied to QR code detection, for a low-cost embedded system with real-time requirements, traditional computer vision techniques combined with optimized decoding libraries often provide a more efficient and robust solution since we have very limited resources to run deep-learning models.

5.1) QR Code Detection Approach

When selecting AI frameworks/techniques here I have considered couple of attributes which are,

1. **Efficiency:** Traditional algorithms are generally less computationally intensive than deep learning models, leading to faster inference times and lower power consumption on the Orange Pi 5 Plus.
2. **Robustness:** Well-established libraries are highly optimized for various lighting conditions, angles, and partial occlusions.
3. **Simplicity:** Easier to implement and debug compared to training and deploying custom deep learning models

Proposed Libraries and Workflow:

1. OpenCV: The OpenCV library will be used for image preprocessing tasks such as:

1. **Grayscale Conversion:** Converting the input image to grayscale to simplify processing.
2. **Thresholding/Binarization:** Converting the image to black and white to enhance QR code features.
3. **Contour Detection:** Identifying potential QR code regions within the image.
4. **Perspective Correction:** Correcting for perspective distortion if the QR code is viewed at an angle, ensuring accurate decoding.

2. ZBar or Quirc: These are highly optimized open-source libraries specifically designed for QR code (and other barcodes) decoding. They are known for their speed and accuracy.

- **ZBar:** A comprehensive library supporting various barcode formats, including QR codes. It's widely used and has Python bindings, making it easy to integrate with ROS2 nodes.
- **Quirc:** A more lightweight and faster QR code decoder, specifically optimized for QR codes. It might require C++ integration within the ROS2 node.

By using the above workflows, we are avoiding any heavy deep learning model inference for our purpose, which is QR code detection, and this approach is suitable for our application. Besides the method mentioned above, we can also use deep learning models like YOLO for more general QR code detection in varied environments.

YOLO (You Only Look Once) Variants (e.g., YOLOv4-tiny, YOLOv5s, YOLOv8n): These models are known for their real-time performance and are well-suited for object detection on embedded devices. The 'tiny' or 'nano' versions are specifically designed for resource-constrained environments.

When considering deploying AI models to the SBC (Orange Pi 5 Plus), we can look at a couple of options, most commonly TensorFlow Lite and ONNX Runtime. I recommend TensorFlow Lite because of its large community support, lightweight inference, and the availability of tools for model optimization.

5.2) Strategies for Performance Optimization (If initial model is too slow for real-time use on the robot)

1. Reduce the model's size and computational load.
 - Quantization: Convert floating-point models to lower precision (e.g., 8-bit integer quantization). This significantly reduces model size and speeds up inference with minimal accuracy loss.
 - Pruning: Remove redundant connections or neurons from the neural network.
 - Knowledge Distillation: Train a smaller model (student) to mimic the behavior of a larger, more complex model (teacher).
 - Architecture Selection: Choose inherently lightweight architectures (e.g., MobileNet, EfficientNet-Lite) designed for embedded devices.
2. Improve Inference Speed while Keeping Accuracy Acceptable:
 - Hardware Acceleration: Leverage the NPU (Neural Processing Unit) or GPU on the Orange Pi 5 Plus (RK3588 SoC) using optimized runtime libraries (e.g., specific TensorFlow Lite delegates for RK3588, or ONNX Runtime with appropriate execution providers). This is the most significant factor for speed improvement.
 - Batching: Process multiple images simultaneously if the application allows for it, though for real-time robotics, single-image inference is often preferred.
 - Optimized Libraries: Ensure that highly optimized versions of libraries like OpenCV and the chosen AI runtime (TensorFlow Lite, ONNX Runtime) are installed and configured to take advantage of the Orange Pi 5 Plus's ARM architecture and NEON instructions.
3. Take Advantage of any Special Hardware Features your SBC Offers:
 - RK3588 NPU: The Rockchip RK3588 SoC in the Orange Pi 5 Plus includes a powerful NPU. Utilizing this NPU through frameworks like RKNN-Toolkit2 (Rockchip Neural Network Toolkit2) or by ensuring the chosen runtime (TensorFlow Lite, ONNX Runtime) has a delegate/execution provider for the NPU would provide substantial speedups for AI inference.

- GPU (Mali-G610): The integrated GPU can accelerate general image processing tasks (e.g., resizing, color conversion) and potentially some deep learning operations if the NPU is not fully utilized or suitable for a specific model.

4. Adjust how Data is Fed into the Model During Operation for Better Performance

- Efficient Data Pipelines: Implement zero-copy or minimal-copy data transfer from the camera driver to the inference engine. Avoid unnecessary data conversions or intermediate buffers.
- Preprocessing on GPU/NPU: Offload image preprocessing steps (e.g., resizing, normalization) to the GPU or NPU if the hardware and framework support it, rather than performing them on the CPU.
- Input Resolution: Optimize the input resolution of the model. Lower resolutions generally lead to faster inference but might reduce accuracy. A balance needs to be found.

5.3) Trade-offs Expected:

- 1) Accuracy vs. Speed/Size: Quantization, pruning, and using smaller models will generally lead to a slight reduction in accuracy. The trade-off is acceptable if the accuracy remains within the operational requirements (e.g., QR code detection still works reliably).
- 2) Latency vs. Throughput: Batching can increase overall throughput but might introduce higher latency for individual inferences. For real-time robotics, low latency is often prioritized.
- 3) Resource Usage vs. Performance: Leveraging hardware accelerators (NPU, GPU) significantly improves performance but might increase power consumption. Optimized models reduce CPU and memory usage.
- 4) Development Complexity: Utilizing specialized hardware features (like the RK3588 NPU) might require using vendor specific tools or more complex configurations, increasing development effort compared to generic CPU inference.

6. OTA Update & Monitoring Flow

6.1) End-to-End Process for OTA Updates

SBC Software Updates (ROS2 Packages, OS)

Update Server: A centralized update server (e.g., a cloud-based server or a local server within the deployment environment) will host the latest software packages and firmware images. This server will be secured and accessible only to authorized personnel.

Version Control: All software components (ROS2 packages, MCU firmware) will be managed under a version control system (e.g., Git). New releases will be tagged and built into deployable artifacts.

Build and Package: New ROS2 packages and system updates for the SBC will be built and packaged into a deployable format (e.g., .deb packages for Ubuntu, or a custom '.tar.gz' archive).

Robot Polling/Notification: The `ota_server_node` on the SBC will periodically poll the update server for new available updates, or the update server can push notifications to the robot when an update is ready.

MCU Firmware Updates (ESP32)

In here we can do the firmware updates to the MCU(ESP32) and those are handled by the SBC through,

1. Firmware Generation: New ESP32 firmware binaries (.bin files) will be generated and stored on the update server.
2. SBC as Proxy: The '`mcu_ota_node`' on the SBC will act as a proxy. When a new MCU firmware update is available, the SBC will download it from the update server
3. Transfer to ESP32: The '`mcu_ota_node`' will then transfer the firmware binary to the ESP32 via the UART interface. The ESP32's built-in OTA bootloader will handle the reception and flashing of the new firmware.
4. Verification and Activation: The ESP32 will verify the integrity of the received firmware (e.g., using checksums) before flashing. After flashing, it will reboot to activate the new firmware. The ESP32 firmware itself will be designed with dual partition support for robust OTA updates, allowing rollback if the new firmware is faulty.
5. Status Reporting: The ESP32 will report the status of the firmware update back to the '`mcu_ota_node`' on the SBC, which in turn will report to the central monitoring server.

Remote Monitoring of Robot Health and Status

The robot's health and status will be monitored remotely using a combination of the existing node architecture (as mentioned above) and standard ROS 2 diagnostic tools. The key idea is to leverage the ROS 2 diagnostics ecosystem to collect, aggregate, and forward health data without introducing entirely new custom nodes.

1. System-Level Health (SBC Monitoring)

- The SBC (Orange Pi 5 Plus) will use the standard '`diagnostic_updater`' package to publish metrics such as CPU load, memory usage, disk utilization, network status, and board temperature.
- Battery voltage and charge percentage from the power system will also be exposed through ROS 2 diagnostics.
- These checks run in the background as part of the SBC's existing ROS stack, not as a separate standalone service.

2. Sensor and Actuator Health

- Each driver node (e.g., *'lidar_driver_node'*, *'imu_driver_node'*, wheel encoder interface on the MCU, camera driver) will integrate with *'diagnostic_updater'* to publish periodic health/status messages (e.g., LiDAR online, IMU calibration status, wheel encoder comms OK).
- The *MCU (ESP32)* already communicates with the SBC. Any errors (e.g., encoder dropouts, motor faults) can be reported through the same ROS diagnostics channel.

3. Diagnostics Aggregation

- On the SBC, the *'diagnostic_aggregator'* package will collect all diagnostic messages (system, sensors, actuators) and generate a unified robot health report.
- This avoids duplicating functionality — we use ROS standard tooling to group and filter health data.

4. Remote Telemetry and Visualization

- The aggregated diagnostics are forwarded to the central monitoring server. This can be done using existing bridges such as *'rosbridge'* (for web dashboards like Webviz) or by exporting metrics into **Prometheus + Grafana** for long-term visualization and alerting.
- Operators can remotely see CPU/battery trends, LiDAR/IMU status, wheel encoder activity, and robot pose (map → odom → base_link) in real time.

7. ROS2 Integration Task

ROS2 Node Design

We need to develop a node(*mcu_data_publisher_node*) that mainly has following responsibilities,

1. **Serial Communication Management:** Establish and maintain a serial connection (e.g., UART) with the MCU.
2. **Data Packet Reception and Validation:** Upon receiving data, the node will be responsible for validating the integrity of the incoming packets.
3. **Data Parsing and Interpretation:** Once a valid data packet is received, the node will parse the raw byte stream into meaningful sensor readings and actuator states.
4. **ROS2 Message Construction:** The parsed data will then be used to populate instances of a custom ROS2 message type, specifically designed to encapsulate all relevant MCU sensor and actuator information.

5. **ROS2 Topic Publishing:** The constructed ROS2 messages will be published to a dedicated ROS2 topic (e.g., /mcu_data).
6. **Error Handling and Logging:** Implement robust error handling mechanisms for serial communication failures, data parsing errors, and any other anomalies.
7. **Parameter Management:** The node will be configurable via ROS2 parameters, allowing for dynamic adjustment of serial port settings (e.g., device path, baud rate) without recompiling the node.

MCU Firmware Changes

1. **Custom Data Packet Definition:** The MCU firmware must define a fixed-size or clearly delimited custom data packet structure. This packet will encapsulate all sensor readings (temperature, humidity, EEPROM min/max) and actuator states (fan speed, button states, bump switch states). A consistent order and data type for each field within the packet are essential.
2. **Data Aggregation and Serialization:** The MCU will need to periodically sample all relevant sensors and read the current states of actuators. This aggregated data will then be serialized into the defined custom data packet format.
3. **Checksum/CRC Calculation:** To ensure data integrity during transmission over the serial link, the MCU firmware should calculate and append a checksum or Cyclic Redundancy Check (CRC) to each data packet. This allows the 'mcu_data_publisher_node' on the SBC to verify the packet's authenticity and detect any bit errors introduced during transmission.
4. **UART/Serial Transmission:** The MCU will be responsible for transmitting these structured data packets over its UART (Universal Asynchronous Receiver-Transmitter) interface at a predefined baud rate. The transmission rate should be chosen to balance data freshness with computational overhead on both the MCU and SBC.
5. **Actuator Command Reception (Optional but Recommended):** While this task focuses on publishing MCU data, a robust system would also allow the SBC to send commands to the MCU (e.g., to set fan speed). This would require the MCU to implement a command reception and parsing mechanism, along with corresponding actuator control logic.

Custom ROS2 Message Type: 'McuData.msg'

```
# McuData.msg Data structure
```

```
std_msgs/Header header
```

```
# Environmental Sensor Data (I2C Sensor)
```

```
sensor_msgs/Temperature temperature
```

```
sensor_msgs/RelativeHumidity humidity
```

EEPROM Min/Max Data

These are read from the MCU, which gets them from its external EEPROM.

float32 min_temperature_recorded

float32 max_temperature_recorded

Actuator Status (PWM Fan)

Fan speed is often represented as a percentage (0.0 to 1.0) or raw PWM value.

float32 fan_speed_percent # Represents the current duty cycle of the fan

Input Button States

bool start_button_pressed

bool stop_button_pressed

bool reset_button_pressed

Bump Switch States

bool front_left_bump_triggered

bool front_right_bump_triggered

bool rear_left_bump_triggered

bool rear_right_bump_triggered

ROS2 Package Design

Package Structure

robot_ws/

```
|— src/
|   └─ mcu_interface/
|       └─ CMakeLists.txt
|       └─ package.xml
|       └─ msg/
|           └─ McuData.msg
|       └─ src/
|           └─ mcu_data_publisher_node.cpp # Or .py if using Python
|       └─ launch/
|           └─ mcu_interface.launch.py
|   └─ config/
|       └─ params.yaml
```

package.xml

```
<package format="3">
  <name>mcu_interface</name>
  <version>0.0.1</version>
  <description>ROS2 package for interfacing with MCU sensor and
actuator data.</description>
  <maintainer email="your_email@example.com">Your Name</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>ament_cmake_python</buildtool_depend>

  <depend>roscpp</depend>
  <depend>roslpy</depend>
  <depend>std_msgs</depend>
  <depend>sensor_msgs</depend>
  <depend>serial</depend> <!-- For Python serial communication -->

  <member_of_group>rosl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(mcu_interface)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(serial REQUIRED) # Assuming a C++ serial library like
boost::asio or a custom one

rosidl_generate_interfaces(
  ${PROJECT_NAME}
  "msg/McuData.msg"
)

add_executable(mcu_data_publisher_node
src/mcu_data_publisher_node.cpp)
ament_target_dependencies(mcu_data_publisher_node rclcpp std_msgs
sensor_msgs ${PROJECT_NAME}__rosidl_generator_cpp serial)

install(TARGETS
  mcu_data_publisher_node
  DESTINATION lib/${PROJECT_NAME}
)

install(DIRECTORY
  launch
  config
  DESTINATION share/${PROJECT_NAME}
)

ament_export_targets(mcu_data_publisher_node HAS_LIBRARY_TARGET)

ament_package()
```

Launch File Design: 'mcu_interface.launch.py'

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    # Get the path to the config file
    config_dir = os.path.join(
        get_package_share_directory('mcu_interface'),
        'config'
    )
    params_file = os.path.join(config_dir, 'params.yaml')

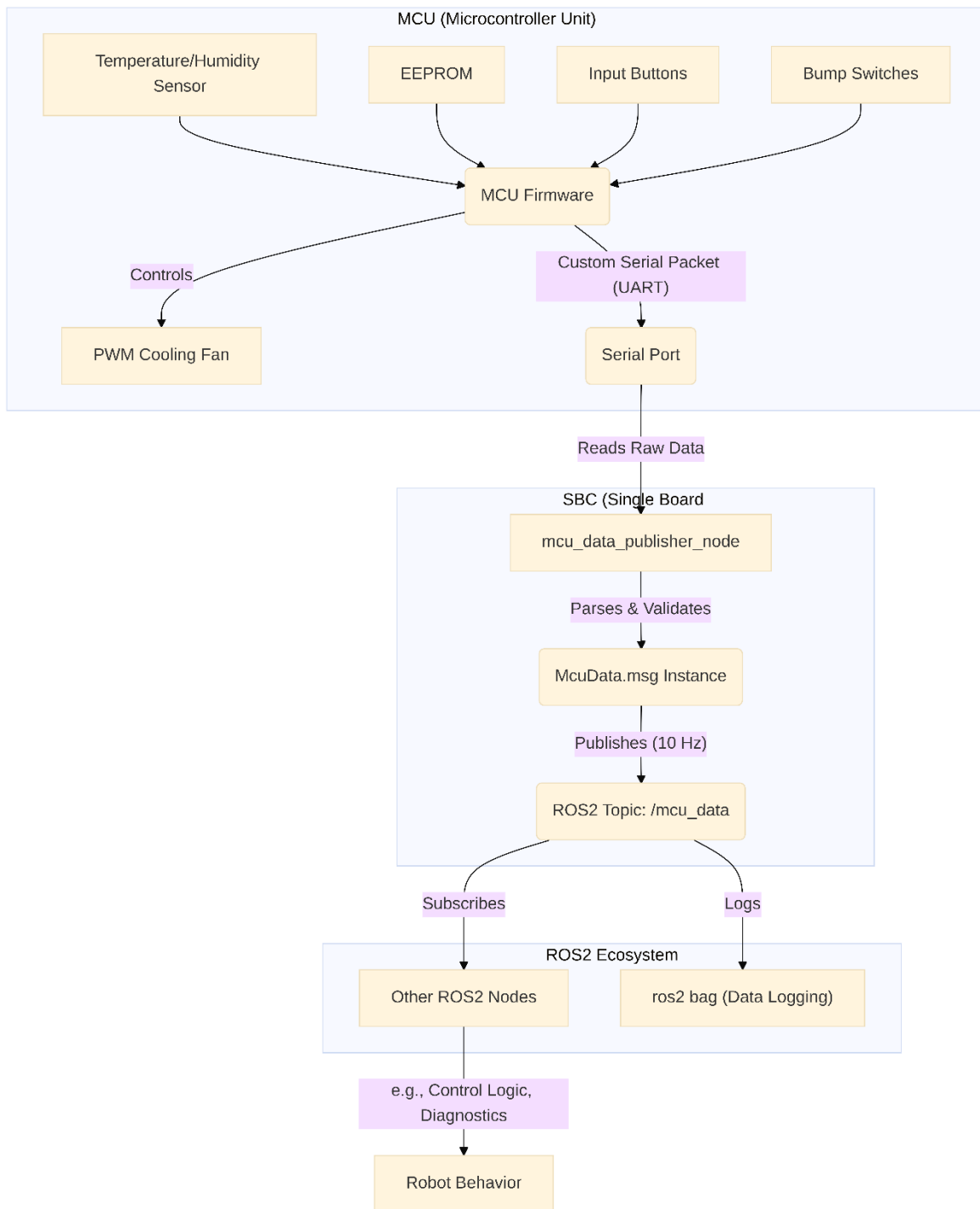
    # Declare launch arguments
    serial_port_arg = DeclareLaunchArgument(
        'serial_port',
        default_value='/dev/ttyUSB0',
        description='Serial port connected to the MCU'
    )
    baud_rate_arg = DeclareLaunchArgument(
        'baud_rate',
        default_value='115200',
        description='Baud rate for serial communication'
    )

    return LaunchDescription([
        serial_port_arg,
        baud_rate_arg,
        Node(
            package='mcu_interface',
            executable='mcu_data_publisher_node',
            name='mcu_data_publisher',
            output='screen',
            parameters=[
                params_file,
                {'serial_port': LaunchConfiguration('serial_port')},
                {'baud_rate': LaunchConfiguration('baud_rate')}
            ]
        )
    ])
```


'params.yaml' File

```
mcu_data_publisher:
  ros__parameters:
    # Default values, can be overridden by launch arguments
    serial_port: /dev/ttyACM0
    baud_rate: 9600
    # Add other node-specific parameters here, e.g., topic name,
publishing rate
    publish_rate_hz: 10.0
```

ROS2 Node Architecture and Data Flow Diagram



Conclude:

In this report, I have used AI tools (DeepSeek, OpenAI) to fix grammar. Used 'kiCAD' as the PCB designing tool.

The attached Google Drive contains all the designs I made using Draw.io, all the code used in this report, and the .png files included within the document and the schematic file.

Link to the GDrive:

<https://drive.google.com/drive/folders/1PrFsNwHKQePNxLUsg4FIElJa5FY1dd?usp=sharing>

Email:

tstrilanka@gmail.com

Phone:

+94 717649512